Compaction in milliseconds

Lab note 15 19 Oct 2026 7 min read
written by Claude from this repository's data

When a long session compacts, Claude Code stops, reads the whole conversation once more and writes a summary of it, and you wait while it writes. In this lab's own session at 436k tokens that wait was 102 s; in a test session at 67k, 15.2 s. smart-compact 0.3.0 writes its summary before it is needed, in the background as a turn ends, so the compaction itself only swaps it in. This note covers how that works, the bug that only a live session showed, what the summary written ahead cost and kept against `/compact`, and why we did not make the summary programmatic.

Where the wait comes from

A compaction is one model request: the conversation in, a summary out. The input is mostly a cache read and costs little time; the summary is output, written token by token, several thousand of them. Nothing else in the session can happen meanwhile.

Claude Code already avoids that wait for its own automatic compaction. Since 2.1.301 it writes that summary well before the auto-compact point, and the compaction then takes about 40 ms and keeps every message since as it is. A mod cannot move that point per session; it is one window for every session. smart-compact compacts much earlier, after a turn, when the compaction pays back within a set number of requests (0.8.0), and there no summary is waiting.

Writing it before it is needed

At a turn's end where a compaction after the turn is due, smart-compact asks a fork of the session for the summary, with its own six-point prompt. A fork reads the session's own cache, so the summary costs what Claude Code's would: repriced on a warm cache, the bench's summaries came to **$0.13** on average ($0.09 to $0.24).

When the compaction comes, smart-compact's own compaction hook answers with the summary, then every message after the last one the fork read, as they are. Claude Code's engine never runs. If the conversation no longer matches what the fork read, the summary is dropped with one line saying why (counts and lengths only), and Claude Code summarizes as before.

The bug only a live session showed

The first live try was in this lab's session, at 436k tokens. The fork wrote its summary in the background, and then the engine summarized anyway: **102 s**. The debug log named the reason in one line: a re-entry, raised by smart-compact itself. A mod's own `$.session.compact` runs through every compaction hook but the calling mod's, so the hook holding the summary never saw the compaction it had asked for. The engine wrote a fresh summary while the one written ahead sat unused.

The 36 unit tests had passed. The test kit calls a mod's hooks directly; it has no notion of which mod raised an event. Two more things only showed up live:

- **A headless session cannot compact from a mod.** `claude -p` answers `$.session.compact` with "not available in a headless session yet". The live checks ran in an interactive session in a detached terminal, with a debug log.
- **The first check never loaded smart-compact.** It depends on statusline; with statusline off, Claude Code skips it with one line in the debug log and nothing in the session.

The fix asks through `/compact`, which Claude Code raises as a manual compaction that every hook sees, smart-compact's included. The transcript shows the summary as one line, whole under ctrl+o.

Live

In interactive sessions at 67k to 70k tokens, on Sonnet 5.5, with compaction after a turn set to fire from 20k tokens:

	Claude Code's summary	Written ahead
Writing the summary	in the compaction: 15.2 s at 67k	in the background after the turn: 16 to 19 s
The compaction itself	15.2 s	3 to 5 ms
A prompt sent before the summary was ready	not tried	compacted in 3.1 ms once it was

The next turn answered from the summary each time. At 436k we have only the engine's 102 s, from before the fix.

Against `/compact`

The compaction bench (`compact_bench.py`) forks a 224k-token session over click. That session read all of `src/click`, was told four decisions about the repository and investigated a bug without fixing it. Each

run compacts, then does one task that needs the files, the conversation or nothing. Six tasks, three runs each, Opus 5.5:

	`/compact`	Written ahead
Runs at full marks	14 of 18	16 of 18
Mean score	0.917	0.981
Context at the next task's first request	42,036 tokens	36,272 tokens
The next task's cost	$0.488	$0.420

The next task cost **−13.9%** [−18.1, −9.4], lower on all six tasks, from −8.6% on the feature to −20.9% on a task that needed nothing from the session. It began on a smaller context (−13.7%) with about as many steps. The score difference, +0.06 [+0.00, +0.18], touches zero: the summary written ahead was not worse, but it was not shown to be better. Where runs fell short: one helper task missed an option's behaviour (2 runs written ahead, 3 with `/compact`), and one rename after `/compact` left the old name in a file (score 0).

The bench measures what the summary keeps, not time. Its summaries were written in line by a headless fork, and the compaction's own cost is not comparable between the arms: 2 of the 18 summaries met a cold cache. Repriced warm, a summary written ahead cost about what Claude Code's own summary costs at that size.

Why not a summary without a model

Output is the dearest token, $20 per million on Opus 5.5. Could Claude instead write one cell that greps, filters and sorts the conversation into a summary, with no summary token written as output?

- **About a fifth of a summary is copying.** In two of Claude Code's summaries from this lab's session (22,136 and 20,173 characters), 17% and 20% of the text repeats the conversation word for word. In 17 of the bench's summaries written ahead (8,535 characters on average) the share was 19%. The rest is judgement: what was decided, what was superseded, where the work stands. A search finds text; it does not decide what still matters.
- **In money, about $0.02.** The session's summaries were about 5.5k tokens of output, about $0.11. Their copied fifth is about $0.02, roughly what the cell's own code would cost in output.
- **In time, nothing.** Writing ahead already hides the time.

The other way to compact with no model is to drop the old tool calls and results and keep the rest. Before its last compaction, this lab's session held, in characters:

Tool results	Tool calls	Prompts	Claude's text	Thinking
46%	34%	12%	6%	2%

Keeping the prompts and Claude's text keeps 18% of the context; a summary keeps about 1%. Every kept token is read again by every later request, so the bigger context costs more to carry than the summary costs to write.

What a kept token costs over its life settles it. Per 1,000 tokens of summary on Opus 5.5: **$0.020** written once as output, $0.008 written to the one-hour cache, then read again by a median of 143 later requests (note 11) at $0.0002 each, $0.029. About **$0.057 over its life**, and output is about a third of it. The lever is a shorter summary, not a cheaper way to write the same one. Claude Code's format re-types every message the person sent (about 3,000 characters in this session's) and pastes code that is already on disk. A leaner summary prompt is the next test; it has not run.

Off until you turn it on

The summary written ahead comes with compaction after a turn (`afterTurnPayback`), and that is off by default. It stays off until it has run in more real sessions. Compacting after a turn is an investment: on short tasks that end right after it, it cost more (0.8.0). With it on, the summary written ahead removes the wait; it does not change when compacting pays.

What this does not show

- **Few live runs.** A handful of compactions, on Sonnet 5.5, at about 70k tokens. The 436k figure is the engine's own, before the fix.
- **Easy tasks.** 18 runs per arm, on tasks both arms mostly solve. The bench can show what a summary loses on these tasks, not on longer work after the compaction.
- **The copied share** comes from two of Claude Code's summaries in one session and 17 bench summaries. It is a measure of overlap with the conversation, not of what a program could have chosen.
- **The lean summary** is a proposal, not a result.

Method & limits

Arms PREP (a fork asked smart-compact's own summary prompt, then `/compact` answered by the bench-only `prep-mod` as smart-compact's hook answers) and COMPACT (`/compact` with no instructions), three prepared slots of about 224k tokens, one run per slot and task: `results-prep.jsonl`. Cost is the next task's own (its main requests and any worker), priced at list prices; the change is for the

average task, with 95% bootstrap intervals that resample runs within each task. The live timings come from Claude Code's debug log of each session; the character shares were counted on the transcripts, in aggregate only.